

Identification of Influential Node Group in Attributed Graph through Explaining Graph Neural Network

Xiao Tan
Southeast University
Nanjing, China
xtan@seu.edu.cn

Tongtong Su
Zhejiang University
Zhejiang, China
12321230@zju.edu.cn

Jiayi Wu
Southeast University
Nanjing, China
220236196@seu.edu.cn

Yan Zhang
Southeast University
Nanjing, China
zhang.yan@seu.edu.cn

Binghui Xu
Southeast University
Nanjing, China
220256505@seu.edu.cn

Dian Shen
Southeast University
Nanjing, China
dshen@seu.edu.cn

Meng Wang
Tongji University
Shanghai, China
mengwangtj@tongji.edu.cn

Beilun Wang*
Southeast University
Nanjing, China
beilun@seu.edu.cn

Abstract

Identification of influential groups of nodes in attributed graphs has applications in a wide range of real-world problems, for instance, collecting important proceedings in citation networks, or identifying essential genes for diagnosing disease in Protein-Protein Interaction networks. Previous approaches for influence maximization manipulated on the graph structure, despite their proliferation, neglect the node attribute information containing additional knowledge. In this work, we introduce Global Graph UNDERstanding (GGUN), a perturbation-based framework leveraging the explanatory power of Graph Neural Networks. It takes into account the entire graph structure and node attributes simultaneously and fuses knowledge through GNN layers. Following the perturbation-based explanation, GGUN fills the gap between Deep Neural Network gradient-based feature importance analysis and discrete structure in the graph, which is formulated as a combinatorial optimization problem. Moreover, GGUN obtains an efficient solution by relaxing the infeasible combinatorial optimization problem with performance guaranteed. Evaluations of synthetic and real-world datasets show that GGUN outperforms baselines on both quantitative metrics and human-intelligible analysis.

CCS Concepts

• Computing methodologies → Neural networks; • Theory of computation → Graph algorithms analysis.

Keywords

Graph neural network, Influential nodes detection, Explanation

*Corresponding Author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792471>

ACM Reference Format:

Xiao Tan, Tongtong Su, Jiayi Wu, Yan Zhang, Binghui Xu, Dian Shen, Meng Wang, and Beilun Wang. 2026. Identification of Influential Node Group in Attributed Graph through Explaining Graph Neural Network. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792471>

1 Introduction

Graphs are widely used to model interacting objects, such as social networks, biomedical interactions, citation networks, and knowledge graphs. One of the fundamental tasks in graph analysis is influential node group detection [29, 33, 46], i.e., identifying a group of the most important nodes in a graph. Influential node group detection is driven by the practical need to understand graph data in various industries. For example, practitioners might market their products to the most influential users for the “word-of-mouth” effect [4, 46], identify key elements for diagnosing neurological disorder [24], or collect important papers in a research field [14].

Many graph theory-based metrics have been introduced in the past to evaluate node importance in complex networks from different perspectives, such as Degree Centrality, Closeness Centrality, Eigenvector Centrality [3, 5, 18, 40], or their combinations. However, these metrics ignore the attribute information in the graph data, namely the oversimplified assumption that all nodes are identical and homogeneous [36]. Furthermore, it is important to consider the coupling effect of the graph data, which may have been overlooked. For instance, p53 is a protein that inhibits the formation of cancer, but its function relies on the interplay among a set of proteins [23]. Likewise, the BRCA1 and BRCA2 genes are associated with breast and ovarian cancer, yet their impacts are not independent [25]. These proteins constitute complexes that exert a joint effect that is not discernible by examining them separately. Therefore, to ensure that all important attribute information and joint influences between different nodes are considered, we make full use of available data, resorting to a data-driven approach, i.e. interpreting machine learning models to describe the characteristics of the data [8, 20].

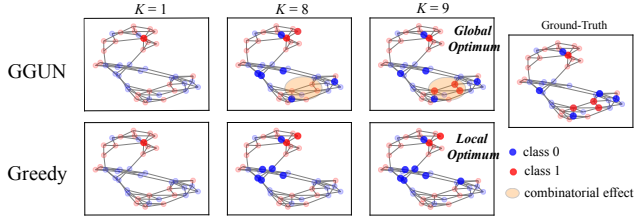


Figure 1: Influential nodes detection on the simulated attributed protein graph containing 2 classes. GGUN can capture the combinatorial effect in the graph and share more similarity with the ground truth, whilst greedy search stuck into local optimum.

We consider investigating the explanatory power of the state-of-the-art models for graphs, namely Graph Neural Networks (GNNs). They jointly model the topological and attribute information of a graph.

Graph neural networks (GNNs) typically rely on both graph structure and node features. While traditional textual encodings such as bag-of-words or TF-IDF (term frequency-inverse document frequency) capture only shallow information, recent works employ advanced language models (e.g., BERT, Sentence-BERT) to generate richer semantic embeddings. However, as these representations become increasingly abstract, it is difficult to understand how they contribute to tasks such as identifying influential node groups. This challenge is further amplified in few-shot scenarios, where labeled data is scarce and interpretability requires stronger evidence. Leveraging the generative ability of large language models (LLMs) to enrich node attributes and augment datasets provides a promising direction, offering both structural and semantic cues for uncovering influential node groups through explaining graph neural network.

There have been abundant studies on explaining deep neural networks (DNNs) [7, 28, 32, 34, 35]. However, the strength of these approaches, beyond the high prediction power of DNN, relies on the ability of precise operations on the continuous data representation [32, 34]. In contrast, GNNs need to simultaneously incorporate **discrete graph structure** and **continuous node features**. For instance, well-known techniques of interpreting DNNs [19, 37, 47] require computing gradients on the features. If applied to the adjacency matrix, which represents the graph topology discretely, they will produce results that cannot be restored to structural information.

In this work, we aim to fill the gap and investigate a proper explanation method that can effectively detect an influential node group for graph understanding. The first thought should be adopting explanation methods that avoid computing gradients over the features. To this end, we consider perturbation-based explanation [38], which interprets a model by perturbing data features and observing its influence on prediction results. However, the existing GNN explanation methods focus on the local explanation of a GNN, i.e., finding the subgraph that has the largest influence on the prediction of one or a group of given nodes. In other words, their outputs are node-specific, and thus unable to find the globally important node group.

To provide a global graph explanation, we aim to select K nodes that, if removed, would change overall node predictions the most. However, this is a combinatorial optimization problem that is computationally challenging due to the rich underlying combinatorial effects in the graph topology. The exhaust approach needs to search the solution in $O(n^K)$ space, where n is the total number of nodes. Since the search space is symmetric with respect to K and $n-K$, we only consider, for convenience of presentation, the case $K \leq n/2$. In this setting, the worst-case complexity is $O(n^{n/2})$. This is an NP-hard problem without certain special conditions. A choice is to adopt the greedy strategy, i.e., adding one node perturbing overall prediction most at each stage [10]. However, this strategy faces the local optimum problem in that the selected nodes tend to be in a small cluster or clique (see Fig. 1). Although much smaller than exhaust search, the search space is still large for real-world large networks. The complexity of the search increases linearly as nodes are selected sequentially, making it infeasible.

In this study, we introduce an effective and efficient approach to provide global graph understanding. Overall, our contributions are:

- We propose GGUN, a **Global Graph Understanding** model through Graph Neural Network perturbation analysis to detect group of influential nodes. GGUN exploits the explanatory power of GNNs to identify a set of nodes that contributes the most to the overall prediction of a node, capturing the potential joint effects of the nodes by formulating a combinatorial optimization problem.
- We design an efficient relaxation to obtain an approximate solution for infeasible combinatorial optimization problem on large networks. After obtaining a continuous solution, we deterministically recover it back to a discrete group of nodes with guaranteed performance. Our method outperforms baselines on this combinatorial optimization problem on both synthetic and real-world datasets. Especially, in the few-shot learning scenario, it shows strong performance on a dataset where a subset of nodes is generated by LLMs.
- We assess the practicality of the selected group of influential nodes in a Protein-Protein Interaction (PPI) network related to a specific cancer. The protein group selected by GGUN has higher diagnostic efficiency and concurrence with survival time than centrality-based methods currently in use, demonstrating the potential of GGUN in real-world networks.

2 Related Work

Graphs are different from images or text in that they have important topological information, which is represented as feature adjacency matrices. Perturbation-based explanation methods on graphs developed from different angles by combining different levels of explanations [44] (structure, attributes or their combination, which all belong to features of the graph) and different graph tasks (node classification, graph classification, link prediction). Our task of influential node detection falls into the category of node classification with node-level explanation.

Most existing methods for node classification explanation focus on providing local explanation [17, 22, 31, 41], i.e., identifying influential factors for one specific node. Specifically, GNNExplainer [41] maximizes the mutual information between the predictions on

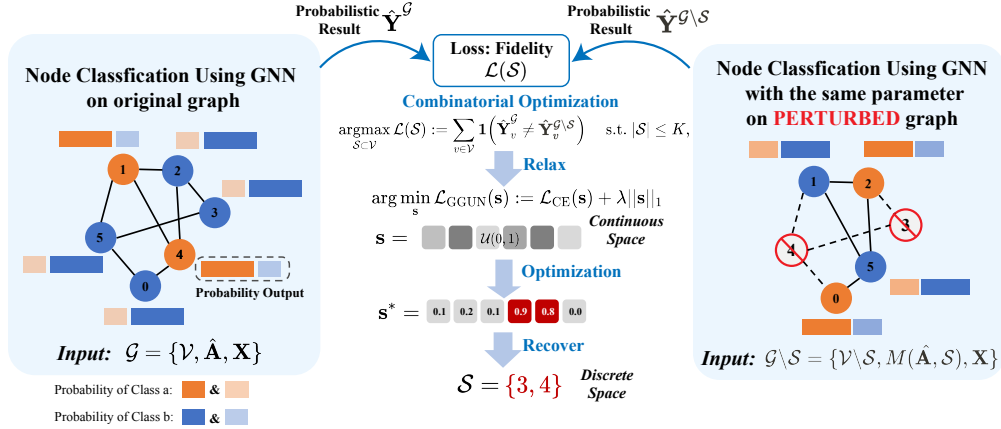


Figure 2: An illustration of the process of identifying the influential node group. (1) First, we estimate a GNN model for the node classification task on the original graph. (2) Then we formulate our perturbation-based explanation objective as a combinatorial optimization problem, which aims to optimize fidelity $\mathcal{L}(S)$ between original output $\hat{Y}^G$ and perturbed output $\hat{Y}^{G \setminus S}$ under sparsity constraint $|S| < K$. (3) Further, we relax infeasible combinatorial optimization into a continuous optimization problem and get optimal solution s^* , and recover s^* back to a discrete node group S .

the original graph and the perturbed graph, while the obtained masks are soft masks. PGExplainer [22], GraphMask [31] learn approximated discrete masks for edges to explain the predictions. GraphLime [17] extends the LIME algorithm to GNN and studies the importance of node features for specific node classification results. However, our task is to provide global explanation, i.e., finding influential nodes for the prediction across the whole graph. A possible way to achieve this is to aggregate the local explanations of all nodes and select the most relevant ones.

3 Preliminary

3.1 Node Classification Using GNN

We consider the task of multi-class node classification in a single attributed graph using GNN. Given a graph $\mathcal{G} = (\mathcal{V}, \mathbf{A}, \mathbf{X})$, the goal is to predict the label of each node $v \in \mathcal{V}$ based on both its attributes and the graph structure, where $\mathcal{V} = \{v_1, \dots, v_n\}$ is a set of n nodes. $\mathbf{A} \in \mathbb{R}^{n \times n}$ is the adjacency matrix and $\mathbf{X} \in \mathbb{R}^{n \times d}$ represents the node attribute matrix, where d is the dimension of the node attributes. There are m classes in total, $\mathcal{C} = \{c_1, \dots, c_m\}$. $\mathbf{Y}^G \in \mathbb{R}^n$ contains labels of n nodes, where $Y_v^G \in \mathcal{C}$ represents the corresponding label for arbitrary node $v \in \mathcal{V}$. The details of the employed GNN model are given in Appendix A.1.1

3.2 Perturbation-based Group Importance Analysis

Perturbation-based explanation [6, 43] is a promising method for studying feature attribution, especially for discrete features in graphs. The intuition is that when important inputs are perturbed, the predictions should differ significantly from the original ones. By perturbing quantitative inputs and measuring the output differences, referred to as **fidelity** [26], it is possible to identify influential factors that contribute to the model's prediction. Perturbation-based explanations ensure precision by specifically defining changes in

the input [9]. If we aim to select a group of features S that jointly have the maximal influence on the prediction, the problem can be formulated as:

$$\begin{aligned} \arg\max_S \quad & \mathcal{L}_{\text{fidelity}}(S) \\ \text{s.t.} \quad & \mathcal{R}(S) \leq 0, \end{aligned} \quad (1)$$

where $\mathcal{R}(S) \leq 0$ is a user-specified constraint depending on which level of explanation they want. There are two components that need to be specified: (1) How to formulate the output difference, so-called fidelity? (2) How to formulate the perturbation constraint, $\mathcal{R}(S)$?

Fidelity A direct formulation of fidelity is the absolute prediction change, i.e., whether the predicted class changes. However, this measure is not sensitive enough to reflect subtle changes in the input and is non-differentiable, which hinders its use in back-propagation necessary for input optimization. To address these issues, various differentiable score functions have been proposed to approximate the absolute prediction change. Two popular choices are Cross Entropy (CE) [12] and Carlini-Wagner (CW) [2], commonly used in adversarial attacks on image classification models. These adversarial attacks aim to perturb pixels subtly, causing the model to misclassify the image. This approach is analogous to our perturbation-based explanation, where the goal is to identify influential features by observing changes in model predictions.

Perturbation Constraint Before discussing the constraints of perturbation, it is essential to define what “perturbing the input” means in the context of graphs. The implementation depends on the level of influential features desired. For our task of detecting an influential node group, we perturb the nodes. However, the formulation of perturbing nodes in a graph is not as intuitive as in other domains. Each node in a graph has two types of information: structural information in the adjacency matrix $\hat{\mathbf{A}}$ and attribute information in the feature matrix $\mathbf{X}$ and label vector $\mathbf{y}$. In this work, perturbing groups of nodes S from the graph $\mathcal{G}$, denoted

as $\mathcal{G} \setminus \mathcal{S}$, refers to **removing** them from the graph, i.e., setting the corresponding rows and columns of $\hat{\mathbf{A}}$ to 0.

$$\mathcal{G} \setminus \mathcal{S} = \{\mathcal{V} \setminus \mathcal{S}, M(\hat{\mathbf{A}}, \mathcal{S}), \mathbf{X}\},$$

$$\text{where } M(\hat{\mathbf{A}}, \mathcal{S})_{i,j} := \begin{cases} 0, & v_i \in \mathcal{S} \text{ or } v_j \in \mathcal{S} \\ \hat{\mathbf{A}}_{i,j}, & \text{otherwise.} \end{cases} \quad (2)$$

Fixing a model parameter, the output probabilistic matrix $\mathbf{P}^{\mathcal{G} \setminus \mathcal{S}}$ will change after perturbing the adjacency matrix. We measure fidelity using $\mathbf{P}^{\mathcal{G} \setminus \mathcal{S}}$ and $\mathbf{P}^{\mathcal{G}}$. However, without any perturbation constraints, the optimal set $\mathcal{S}$ would naturally become the entire node set in the graph, since removing all nodes would have the greatest impact on the prediction. This result is meaningless and provides no useful information to the user. Good explanations should be sparse [44] and accurate, meaning that they should highlight the most important features while ignoring irrelevant ones. In our work, we limit the size of the selected node group to $|\mathcal{S}| \leq K$. When we aim to select exactly K nodes, this can be achieved after continuous relaxation and soft constraints by tuning the corresponding hyperparameters.

Then we present a formal mathematical definition of the influential node group discussed in this paper:

DEFINITION 3.1 (INFLUENTIAL NODE GROUP IN NODE CLASSIFICATION TASK). Given a node classification model f trained on graph $\mathcal{G} = (\mathcal{V}, \mathbf{A}, \mathbf{X})$, and let $\hat{\mathbf{Y}}_v^{\mathcal{G}} = f(\mathcal{G}, v)$ be the predicted label of node $v \in \mathcal{V}$. The influential node group $\mathcal{S} \subset \mathcal{V}$ is defined as a set of nodes whose removal maximally alters the predictions of the GNN model and $|\mathcal{S}| \leq K$. This can be expressed mathematically as finding a set $\mathcal{S}$ that maximizes the change in prediction accuracy:

$$\operatorname{argmax}_{\mathcal{S} \subset \mathcal{V}, |\mathcal{S}| \leq K} \Delta(f(\mathcal{G}, v), f(\mathcal{G} \setminus \mathcal{S}, v)) = \sum_{v \in \mathcal{V}} \mathbf{1}(\hat{\mathbf{Y}}_v^{\mathcal{G}} \neq \hat{\mathbf{Y}}_v^{\mathcal{G} \setminus \mathcal{S}}) \quad (3)$$

where Δ measures the differences in GNN predictions before and after the removal of the nodes in $\mathcal{S}$ from the graph $\mathcal{G}$.

4 Method

We design the proposed method GGUN, leveraging both node attributes and graph structure information in the following steps: (1) We start by estimating a node classification model on the original graph (Section 3.1). (2) Following the idea of perturbation-based explanation, i.e., the output will change significantly once important features are perturbed, we formulate the combinatorial optimization problem (Section 4.1). (3) We further relax the combinatorial optimization with infeasible searching space into continuous optimization. After finding the optimal solution $\mathbf{s}^*$ in continuous space, we recover it back to a discrete solution in the form of a node group $\mathcal{S}$ (Section 4.2). Fig. 2 shows the workflow of our method.

4.1 Perturbation-based Explanation for GNN

Our objective function follows the perturbation-based explanation schema: **remove a node group $\mathcal{S} \subset \mathcal{V}$ to maximize the prediction change rate across the whole graph**. We use $\mathcal{G} \setminus \mathcal{S}$ to represent the rest of $\mathcal{G}$ after removing the perturbed nodes. It can be written as (2).

We set the corresponding rows and columns in the adjacency matrix to 0, which consequently influences the feature propagation process in the GNN model.

The above schema is equivalent to maximizing the fidelity, i.e., the number of changed predicted class labels. The perturbation constraint is the group size. With $\mathbf{1}(\cdot)$ as the indicator function, we formulate the objective function:

$$\operatorname{argmax}_{\mathcal{S} \subset \mathcal{V}} \mathcal{L}(\mathcal{S}) := \sum_{v \in \mathcal{V}} \mathbf{1}(\hat{\mathbf{Y}}_v^{\mathcal{G}} \neq \hat{\mathbf{Y}}_v^{\mathcal{G} \setminus \mathcal{S}}), \quad (4)$$

$$\text{s.t. } |\mathcal{S}| \leq K,$$

where $\hat{\mathbf{Y}}_v^{\mathcal{G} \setminus \mathcal{S}}$ is the predicted labels of node v on the perturbed graph. However, since the objective function contains an indicator function, the objective function is non-differentiable. Furthermore, the output is insensitive to minor modifications to the graph. Because the objective function is a measure of the performance of the whole graph, whereas minor changes to the nodes in $\mathcal{S}$, such as adding or deleting one, may only affect $\mathcal{G} \setminus \mathcal{S}$ locally, without significantly affecting the structure of the whole graph. Therefore, we use a continuous probabilistic form to approximate the hard prediction change in (4). Previous works have demonstrated that the Cross-Entropy (CE) loss is a suitable approximation, which extends the node classification training loss. The perturbation-based explanation methods use the CE loss function to minimize the probability of the original predicted class. Thus, we can rewrite (4) as follows:

$$\operatorname{argmin}_{\mathcal{S} \subset \mathcal{V}} \mathcal{L}_{\text{CE}}(\mathcal{S}) := \sum_{v \in \mathcal{V}} \sum_c \left(\mathbf{1}(\hat{\mathbf{Y}}_v^{\mathcal{G}} = c) \cdot \ln \mathbf{P}_{v,c}^{\mathcal{G} \setminus \mathcal{S}} \right) \quad (5)$$

$$\text{s.t. } |\mathcal{S}| \leq K.$$

where $\mathbf{P}^{\mathcal{G} \setminus \mathcal{S}}$ is the output probabilistic matrix of perturbed graph $\mathcal{G} \setminus \mathcal{S}$ through forward propagation. Note that the original class is not necessarily the same as the ground-truth, since the fitting is not likely to be completely accurate.

The problem in (5) is a combinatorial optimization problem with a search space of approximately $O(n^K)$. Consequently, we face a challenge of perform an exponentially large number of node classification tasks, resulting in immeasurable computational complexity. To address this challenge, we consider relaxing the discrete variable $\mathcal{S}$ into continuous variable $\mathbf{s} = (s_1, \dots, s_n)^\top$, where $s_i \in [0, 1]$. Therefore the variable can be directly optimized via backpropagation. We can regard s_i as the probability that node v_i should be removed. According to (2), the graph perturbed by $\mathbf{s}$ can be formulated as:

$$\mathcal{G}' = \{\mathcal{V}, M'(\hat{\mathbf{A}}, \mathbf{s}), \mathbf{X}\}, \quad (6)$$

$$\text{where } M'(\hat{\mathbf{A}}, \mathbf{s}) = \text{diag}(\mathbf{1} - \mathbf{s}) \hat{\mathbf{A}} \text{diag}(\mathbf{1} - \mathbf{s})$$

and (5) can be reformulated into a problem with continuous variable:

$$\operatorname{argmin}_{\mathbf{s}} \mathcal{L}_{\text{CE}}(\mathbf{s}) := \sum_{v \in \mathcal{V}} \sum_c \left(\mathbf{1}(\hat{\mathbf{Y}}_v^{\mathcal{G}} = c) \cdot \ln \mathbf{P}_{v,c}^{\mathcal{G}'} \right) \quad (7)$$

$$\text{s.t. } \|\mathbf{s}\|_1 \leq K'$$

It is difficult to optimize a loss function with a hard constraint of $\|\mathbf{s}\|_1 \leq K'$ in practice. Additionally, the relationship between K and K' is hard to formalize after relaxation. Therefore, we include the constraint as one regularization term in the objective function (7), and it becomes a soft constraint:

$$\underset{\mathbf{s}}{\operatorname{argmin}} \mathcal{L}_{\text{GGUN}}(\mathbf{s}) := \mathcal{L}_{\text{CE}}(\mathbf{s}) + \lambda \|\mathbf{s}\|_1, \quad (8)$$

where λ implicitly determines the level of sparsity. Even though there is not a closed-form equation between hard and soft constraint, there exists a λ that is equivalent to some K for the hard constraint. In practice, the users need to use tuning parameters to find an appropriate sparsity level. This process is not always continuous, since multiple entries in $\mathbf{s}^*$ may be selected at once with a small decrease in λ due to the use of Lagrange multipliers. Although λ can be adjusted freely, there is an optimal range for it. When λ is too large, the score function prioritizes sparsity over fidelity, resulting in poor optimization for fidelity. However, sparsity and fidelity are not mutually exclusive, and both terms of the score function can be reduced together with λ in the optimal range.

Then we can recover $\mathbf{s}$ to a discrete space according to the equivalence relationship between $\mathcal{S}$ and $\mathbf{s}$:

$$\mathbf{1}_{[\epsilon, 1]}(s_i) = 1 \Rightarrow v_i \in \mathcal{S} \quad (9)$$

where ϵ is a hyperparameter, which can also control the sparsity level: the higher ϵ , the smaller size of $\mathcal{S}$.

4.2 Optimization

4.2.1 Optimization in Continuous Space. So far, we have transformed the combinatorial optimization problem into a continuous one, which can be solved using **derivative-based optimization**. Although there is still a constraint, $\mathbf{s} \in [0, 1]^n$ is more manageable than $\mathcal{S} \in \{0, 1\}^n$. Projected Gradient Descent (PGD), is a standard way to solve the optimization problem in (9) with constraint. It ensures that the point arising from the gradient should lie in the feasible set by taking projection operation after each gradient step:

$$\mathbf{s}^t = \Pi_{[0, 1]^n}(\mathbf{s}^{t-1} - \eta \nabla_{\mathbf{s}} \mathcal{L}_{\text{GGUN}}(\mathbf{s}^{t-1})). \quad (10)$$

However, the objective function is non-convex and challenging to analyze due to the high dimensionality of the problem, leading to a high risk of getting trapped in a local optimum during optimization. To mitigate this, we can randomly initialize multiple starting points to improve the chances of discovering global optima.

Recent advances of **derivative-free optimization** [16, 27] allow efficiently approximating the global optimal solutions of arbitrary functions. One typical method in this field is classification-based optimization, SRACOS [16]. Specifically, it first uniformly sample r points from the whole space, then queries the objective function to assess the generated solutions and forms a binary classification dataset $\mathcal{B}_0 = \{(\mathbf{s}^1, \mathcal{L}_{\text{GGUN}}(\mathbf{s}^1)), \dots, (\mathbf{s}^r, \mathcal{L}_{\text{GGUN}}(\mathbf{s}^r))\}$. Second, it learns a classification model to partition the solution space into good area $\mathcal{D}_0$ and bad area $[0, 1]^n \setminus \mathcal{D}_0$. The model is an axis-parallel rectangle to cover all positive solutions while avoiding negative solutions, while positive or negative is determined through comparison of objective scores. Then the new solutions are sampled from the good areas with probability p , otherwise from the whole space:

$$\mathbf{s}^{r+1} = \begin{cases} \text{Sampling } (\mathcal{U}_{\mathcal{D}_0}) & \text{w.p. } p \\ \text{Sampling } (\mathcal{U}_{[0, 1]^n}) & \text{w.p. } 1 - p \end{cases} \quad (11)$$

We add $\mathbf{s}^{r+1}$ to the classification dataset: $\mathcal{B}_{r+1} = \mathcal{B}_r \cup \{(\mathbf{s}^{r+1}, \mathcal{L}_{\text{GGUN}}(\mathbf{s}^{r+1}))\}$, and begin the next loop of optimization. The good area will shrink

Table 1: Comparison of F-Acc, F-Prob on Cora. For GGUN, the selected group size K changed with different λ : When $\lambda = 4$, $K = 238$. We unify $K = 238$ for fair comparison with baselines.

Metric	GCN-fixed θ		GCN-new		GAT		Coverage($\uparrow$)
	F-Prob($\downarrow$)	F-Acc($\downarrow$)	F-Prob($\downarrow$)	F-Acc($\downarrow$)	F-Prob($\downarrow$)	F-Acc($\downarrow$)	
Random	0.8497	0.8804	0.9486	0.9798	0.9305	0.9453	—
Degree	0.6921	0.7463	0.9408	0.9667	0.8983	0.9175	0.30
Eigenvector	0.8523	0.8689	0.9476	0.9785	0.9242	0.9399	0.15
Closeness	0.8195	0.8445	0.9431	0.9659	0.9071	0.9262	0.37
GNExplainer-TE	0.7905	0.8176	0.9391	0.9635	0.8840	0.9065	0.27
PGExplainer	0.8579	0.8756	0.8312	0.8445	0.9255	0.9465	0.25
GraphLIME	0.9408	0.9542	0.8924	0.8992	0.9636	0.9790	0.25
Gradient	0.8474	0.8752	0.9471	0.9707	0.9093	0.9273	0.34
EHCC	0.8089	0.8252	0.9413	0.9672	0.8965	0.9151	0.27
KPDN	0.9595	0.9620	0.9524	0.9843	0.9725	0.9849	0.37
Greedy	0.8586	0.8220	0.9515	0.9830	0.9367	0.9500	0.25
GGUN	0.6707	0.7290	0.9332	0.9635	0.8478	0.8765	0.53

during optimization. The optimization quality depends on the number of training loops, which is proportional to the number of comparisons, and the number of queries to the objective function.

4.2.2 Recover to Discrete Space. After obtaining optimal solution $\mathbf{s}^* \in [0, 1]^n$, we need to further recover it back to discrete node group $\mathcal{S}$. If not, the relaxing operation also breaks the discrete property of the graph nodes, facing the same problem with gradient-based methods. We propose two recovery algorithms: Hard Threshold and Compared Rounding.

Hard Threshold is a direct recovery method according to the equivalence in (9), which involves selecting a threshold value ϵ and rounding all values above the threshold to 1 and all values below the threshold to 0:

$$\Pi_{\{0, 1\}^n}^{\text{hard}}(\mathbf{s}_i^*) := \mathbf{1}_{[\epsilon, 1]}(s_i). \quad (12)$$

We set ϵ to 0.5 as an intuitive way to round $\mathbf{s}^*$ to the nearest binary points, $\underset{\mathbf{s} \in \{0, 1\}^n}{\operatorname{argmin}} \|\mathbf{s} - \mathbf{s}^*\|$. Hard Threshold is simple and fast but may result in a suboptimal solution since it does not consider the optimization principles in the first place. It eliminates the concept of probability for entries in $\mathbf{s}$. For instance, both $s_i = 0.6$ and $s_i = 0.9$ would be set to 1, which leads to information loss.

Compared Rounding[39], on the other hand, rounds entries in $\mathbf{s}^*$ to either 0 or 1 based on optimization objectives:

$$\Pi_{\{0, 1\}^n}^{\text{round}}(\mathbf{s}_i^*) := \underset{j \in \{0, 1\}}{\operatorname{argmin}} \mathcal{L}_{\text{GGUN}}(\mathbf{s}_{1:i}^*, j, \mathbf{s}_{i:n}^*). \quad (13)$$

Compared Rounding tends to generate better solution compared with Hard Threshold, although they take more time during comparison due to two additional forward steps per entry. Although it incurs additional computational costs, the efficiency advantage over greedy search is significant enough to offset it.

5 Experiment

In this section, we provide empirical insights into the effectiveness of our GGUN. The experiments are aimed at supporting the following two claims:

Claim 1: Superior optimization performance Within the framework of perturbation-based explanation, GGUN is capable of discovering superior solutions for the combinatorial optimization problem outlined in equation (4) compared to baseline methods.

Table 2: Comparison of F-Acc, F-Prob on Cora-LLM4NG. For GGUN, the selected group size K changed with different λ : When $\lambda = 3$, $K = 209$. We unify $K = 209$ for fair comparison with baselines.

Metric	GCN-fixed θ		GCN-new		GAT	
	F-Prob($\downarrow$)	F-Acc($\downarrow$)	F-Prob($\downarrow$)	F-Acc($\downarrow$)	F-Prob($\downarrow$)	F-Acc($\downarrow$)
Random	0.8053	0.8895	0.8246	0.9024	0.7700	0.9719
Degree	0.6257	0.7459	0.7167	0.8215	0.7502	0.9316
Eigenvector	0.7790	0.8686	0.8105	0.8909	0.7689	0.9759
Closeness	0.7567	0.8366	0.7827	0.8704	0.7611	0.9561
GNNExplainer-TE	0.6938	0.7945	0.7547	0.8456	0.7520	0.9582
PGExplainer	0.7571	0.8747	0.8000	0.9006	0.7712	0.9798
GraphLIME	0.7852	0.8747	0.8289	0.9082	0.7866	0.9914
Gradient	0.6924	0.7768	0.7415	0.8287	0.7561	0.9424
EHCC	0.6568	0.7588	0.7264	0.8254	0.7518	0.9370
KPDN	0.8506	0.9467	0.8545	0.9334	0.7899	0.9996
Greedy	0.6166	0.7425	0.7081	0.8099	0.7616	0.9359
GGUN	0.6140	0.7397	0.7010	0.8063	0.7522	0.9341

Claim 2: Truthful to ground-truth knowledge The node group selected by GGUN is closer to the ground-truth knowledge about the influential node group.

5.1 Claim 1: Superior optimization performance

5.1.1 Settings. Dataset: We use the well-known citation network Cora[1] and its , which contains 2708 nodes representing publications and 5429 links representing one publication citing another one. The node feature dimension is 1433, equivalent to the one-hot embedding of keywords from the abstract. In addition, we construct an enhanced version of the dataset, named Cora-LLM4NG, by applying the LLM4NG[42] method to augment the original labeled data. Specifically, Cora-LLM4NG adds 21 new labeled nodes and 42 additional edges to the original graph, where only 5 nodes were labeled before augmentation. We also conduct experiments on two larger datasets to validate the scalability of our method, with results omitted due to space limitations.

Quantitative Metrics: To verify Claim 1, we take into account both the fidelity and sparsity of the GGUN objective function CE-softmax(see appendix A.1.2) throughout the evaluation. We compare the fidelity scores at the same level of sparsity (K), while fidelity (F for short) can be evaluated in two ways: F-Prob (Prob being short for probability, as CE-softmax is based on probabilistic matrix $\mathbf{P}^G$) and F-Acc (Acc being short for accuracy, which represents the change of the absolute prediction).

- F-Prob: $= -\frac{1}{n} \sum_{v \in \mathcal{V}} \sum_c \left(\mathbf{1}(\hat{\mathbf{Y}}_v^G = c) \cdot \mathbf{P}_{v,c}^{G \setminus S} \right)$
- F-Acc: $= \frac{1}{n} \sum_{v \in \mathcal{V}} \mathbf{1}(\hat{\mathbf{Y}}_v^G = \hat{\mathbf{Y}}_v^{G \setminus S})$.

For both metrics, lower values indicate better optimization performance. Two metrics are highly relevant, while F-Prob is more sensitive than F-Acc.

Baseline methods And Implementation details We selected methods: Centrality-based methods (Degree, Eigenvector, and Closeness), GNNExplainer, PGExplainer, GraphLIME, Gradient, Greedy, EHCC, KPDN as baselines, with detailed descriptions and implementation details provided in the appendix A.2.

5.1.2 Results. Fidelity Table 1 and Table 2 show the performance of two versions of fidelity tested on three GNN models, on the Cora and Cora-LLM4NG datasets, respectively. Statistical significance

tests are conducted with all p-values below 10^{-4} , with details omitted due to space constraints. Not surprisingly, We observe that even though the node classification and influential node detection are manipulated on one GCN model, removing those selected nodes also changes the prediction results significantly on GAT or another GCN model with a different initialization seed and model parameters. GGUN still maintains significant advantages over baselines, suggesting that our selected group of influential nodes is independent of the specific GNN model employed.

Distribution Besides fidelity, the distribution of the selected nodes should also be taken into consideration. Although the ground-truth knowledge of the most influential papers in Cora is unrevealed, the distribution of selected nodes can help us identify the rationality of the methods. Intuitively, the selected group of nodes should scatter across the whole graph for global understanding. We further analyze the distribution from two aspects: **Graph space:** The visualization effect largely depends on the layout algorithm. We employ the well-established balloon layout algorithm from yed-live, which reads graphML format files and arranges nodes automatically. As shown in Fig. 3, nodes selected by all GNN perturbation-based methods (GGUN, Greedy, Gradient, GNNExplainer) scatter across clusters in the graph, while nodes selected by (degree) centrality concentrate in the largest cluster. **Representation space:** After node classification estimation, the hidden layer representation can be regarded as a space considering structure and node features simultaneously. We also adopt t-SNE to support visualization. As shown in Fig. 4, nodes selected by GGUN dispersed in space more evenly than those of baselines. Further, we grid the space by 10×10 (as shown in Fig. 4(a)) and calculate the coverage rate of those selected nodes. As revealed in the Coverage column in Table 1, GGUN outperforms baselines significantly with the highest coverage rate of 0.53.

5.2 Claim 2: Truthful to ground-truth knowledge

Influential nodes in the graph are not as intuitive as influential pixels in the image, i.e., the saliency map. Therefore, additional knowledge about the dataset is required. It is challenging to select ground-truth influential papers in the Cora dataset. Noted that high-citation papers are not necessarily high-influential papers, individuals may hold subjective views on the interpretation of being influential and pick out papers in line with their own interests.

There are two directions for substitute solutions: (1) generating a synthetic dataset equipped with prior knowledge. The rules of generating a dataset can be regarded as reasonable approximations of the ground truths. (2) In the field of bioinformatics, there already exists large amounts of research and knowledge about the ground-truth influential group. Therefore, the challenges mainly emanate from two actions, namely the collection of information from an external database and further estimation upon properly supervised tasks that data-driven methods require. In the following experiments, we prepare a dataset from both two directions. The definition of accuracy depends on the dataset generation. We introduce the definition of **accuracy** following the dataset generation of each.

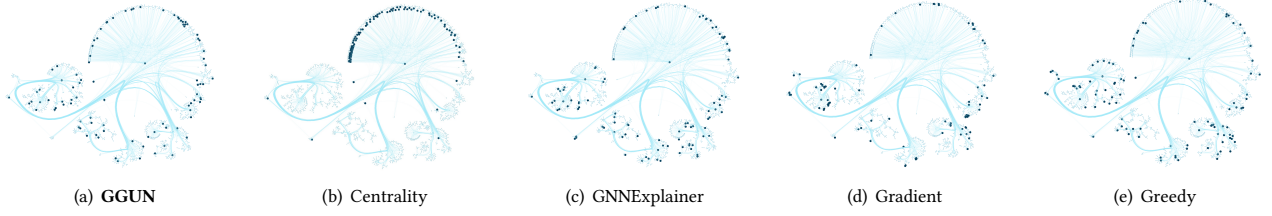


Figure 3: Visualization of Top-100 selected nodes in Cora with balloon layout. It shows that GNN perturbation-based method(GGUN, Greedy, Gradient, GNNExplainer) scatters across all the clusters in the graph. In contrast, nodes selected by degree centrality only concentrate in the largest cluster.

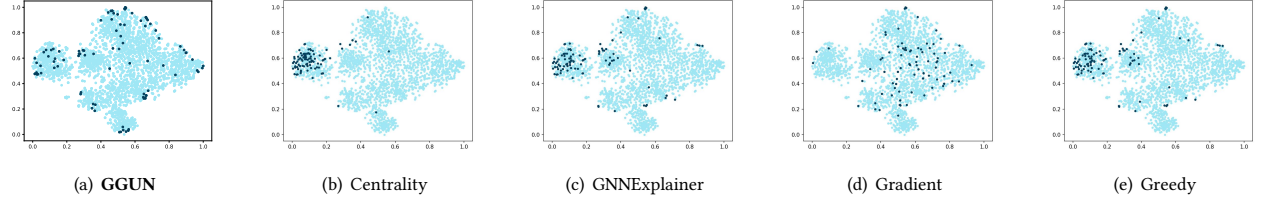


Figure 4: Visualization of Top-100 selected nodes in Cora in hidden space ($d_{\text{hidden}} = 16$). We use t-SNE to further reduce the dimension to $d = 2$ and normalize the scale to $[0, 1]$. Grid (as shown in (e), divides the space into 100 blocks) is used to quantitatively measure the Coverage rate ($\uparrow$) of selected nodes. Nodes selected by GGUN disperse in space more evenly than baselines.

Given that GGUN has demonstrated the best optimization quality among GNN perturbation-based methods, we only need to compare it with centrality-based methods to verify claim 2. We evaluate GGUN with an additional baseline, Influence Spread with the Independent Cascade (ISIC) in this section. Since GGUN and Influence Spread are two different frameworks, the evaluation metric is narrowed to Accuracy only. For the hyperparameters ISIC required, we set activation probability 0.2, spread steps 1000 as default.

5.2.1 Synthetic Dataset. Dataset generation: We generate 2 synthetic attributed graphs for the case study. Considering the scenario of clustered nodes, each cluster includes center nodes. The ground-truth influential nodes should cover all clusters. For example, a social network with many hobby communities, or a paper citation network with different topics. The size of clusters differs from each other, and there might be some large clusters. Traditional graph centrality-based methods only focus on those large clusters and ignore center nodes in other clusters.

Based on this observation, we generate a graph with c clusters, each containing nodes belonging to one class. The cluster size is denoted as $\mathbf{n} = \{n_1, n_2, \dots, n_c\}$, where n_c is much greater than others. We arrange center nodes within each cluster that link to all other nodes within the same cluster, and these center nodes are considered the ground-truth influential group of nodes(as seen in Fig.5). The number of the center nodes are $\mathbf{m} = \{1, 1, \dots, c-1\}$. We randomly append e edges across the whole graph to simulate cross-cluster edges. The feature generation for different classes follows the multivariate normal distribution. We generate two case studies: **Case 1:** $N = 100$, $\mathbf{m}/\mathbf{n} = \{1/15, 1/15, 1/15, 1/15, 4/40\}$, $e = 0$ (Fig. (5) visualize the case). **Case 2:** same as Case 1, but $e = 10$.

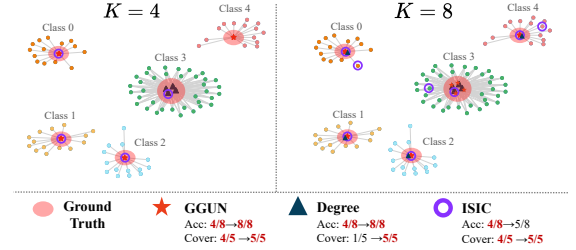


Figure 5: Visualization of identified influential nodes with different methods. The centrality-based method is attracted by the largest cluster with only a 1/5 Coverage rate, ignoring the other 4 clusters, while GGUN with its global understanding of the graph has a 4/5 Coverage rate.

Accuracy evaluation: Prior knowledge of the ground-truth influential nodes is derived from the data generation process: the centering 8 nodes scatter across all clusters. Thus, we employ the accuracy $\rightarrow 8$ to evaluate the rationality of methods. When K is small, the selected nodes should cover more clusters for a global understanding of the whole graph. We employ the Coverage rate $\rightarrow 5$ as an auxiliary metric accordingly.

Table 3 shows the performance of GGUN and baselines of 2 cases. For both cases, we change λ to get two groups of nodes with different K . When $K = 4$, all methods reach 4/8 accuracy, while the coverage rate of the centrality-based method is 1/5 only. As shown in Fig. 5, centrality only focuses on the largest cluster. It is worth mentioning that for GGUN, with continuous decreasing λ from 0.3 to 0.2, the K increases from $K = 4$ to $K = 8$: the 4 center nodes in

Table 3: Accuracy($\uparrow$) and Coverage rate($\uparrow$) of two synthetic cases. For GGUN, when decreasing λ from 0.3 to 0.2, 4 center nodes of class 4(in Fig. 5) are selected simultaneously, which verifies that GGUN can capture the joint effect in the graph when tuning λ .

Metric	Acc($\uparrow$)	Cover($\uparrow$)	Acc($\uparrow$)	Cover($\uparrow$)
	$K = 4$		$K = 8$	
Degree	4/8	1/5	8/8	5/5
Eigenvector	4/8	1/5	4/8	1/5
Closeness	4/8	1/5	4/8	1/5
ISIC	4/8	4/5	5/8	5/5
GGUN	4/8	4/5	8/8	5/5

the largest cluster are selected simultaneously (from left to right in Fig. 5), which verifies the joint effect in the graph. Such an effect cannot be detected by all the baselines. Although ISIC has the same coverage rate as GGUN, when $K = 8$, the accuracy of ISIC is lower. The failure to select 4 centering nodes together means that the joint effect cannot be captured by ISIC.

5.2.2 Protein-Protein Interactions network. Dataset generation: Protein-protein interaction (PPI) networks are physical contacts of high specificity established between protein molecules as a result of biochemical events. Since genes encode proteins and proteins dictate cell function (cancer), identifying the determined group of genes for specific cancers can be transferred to the task of identifying the influential group of proteins. For a biological disease, differentially expressed genes (DEGs)[30] are those with observed differences in expression levels between patients and controls, revealed through statistically significant analysis. The significant genes include upregulated(+) and downregulated(-) genes. Concretely, we focus on two typical cancers, glioblastoma(GBM) and colorectal(COAD).

The workflow of GGUN influential genes detection on PPI is revealed in Fig. 7. The first two steps of data processing are the same as traditional workflows of influential gene detection task [3]. **Identification of DEGs:** DEGs are identified through limma differential analysis[30] on GEO database, which contains gene expression samples of the cancer patient and control healthy human. We set the threshold as $|\log FC| > 1$, $P < 0.05$, and collect corresponding unregulated(+) and downregulated(-) genes. The statistical characteristics of the two cancers are included in Table 4. **Constructing attributed graph:** We put the collected DEGs into STRING database to construct PPI network with 9-dimension edge features. Procedures so far are in the same way as a traditional PPI hub gene analysis work. The traditional method utilizes centrality (for the most time use degree) to select top- K rank nodes.

The next step is tailored for GGUN: **Node classification task:** We average across all attached edges to get corresponding node features referring to the implementation of [15], with nodes features being 9-dimension accordingly. One can choose whatever classification task once they find the proper label of proteins from another external database. Specifically, for our work, we formulate the task as a natural binary classification task with the goal of predicting one protein to be upregulated or downregulated. Since we are not using a benchmark, we first manage to guarantee the learnability

Table 4: Node classification performance on two PPI datasets. The train-val split is 0.8:0.2, and a 2-layer graph convolutional network (GCN). The estimation performance is sufficient to demonstrate that the task can be effectively learned.

Cancer	#nodes	#(+)	#(-)	Accuracy	Precision
Glioblastoma	378	240	138	0.8267	0.7500
Colorectal	70	22	48	0.8571	0.7895

Table 5: Diagnostic efficiency($\uparrow$) of two cancers. GGUN performs the best or the second best.

Cancer	Glioblastoma			Colorectal		
	$K = 4$	$K = 19$	$K = 38$	$K = 1$	$K = 4$	$K = 8$
Degree	0.7884	0.8269	<u>0.9230</u>	0.5824	<u>0.6923</u>	0.7582
Eigenvector	0.6602	0.8076	<u>0.9230</u>	0.4945	0.6703	0.7472
Closeness	0.7884	0.8653	0.9038	0.5824	0.6813	<u>0.7802</u>
ISIC	0.7884	0.8269	0.9038	0.5824	0.6813	0.7582
GGUN	0.7884	<u>0.8461</u>	1.0000	0.5824	0.8461	0.9010

of the classification task. Specifically, we split train:val as 0.8 : 0.2. The classification performance is revealed in Table 4. Although the performance doesn't reach the level of well-known benchmarks in node classification tasks, we think the task is learnable enough to capture the underlying information of the PPI graph and support further GGUN analysis.

Accuracy evaluation: According to previous work [3], we validate whether the selected influential genes are practically essential through **diagnostic accuracy** on another patient clinical database: TCGA, which contains genome expression over 60000 genes for 20 kinds of cancers. We only employ the selected groups of genes as features to generate predictions. For diagnostic accuracy, we construct a balanced binary classification task between the targeted cancer (GBM and COAD) and another different cancer with a comparable number of samples(GBM(160)/ESCA(165), COAD(214)/KIRC(187)). The ratio of train-val split is 0.8 to 0.2.

Table 5 shows the results. GGUN generally achieves the best or second-best scores among all the options. For relatively small values of K , most methods achieve high diagnostic accuracy. We suggested when K is large, there may be some joint effects between nodes. However, such rich joint effects cannot be captured by other methods except GGUN. Thus, we propose GGUN to be the most reliable choice since GGUN reveals better overall performance.

Acknowledgements

This work was supported by National Natural Science Foundation of China under Grant No. 625B2048, 61906040, 61972085, 62576095, 62276063, 62522205, 62272101, Southeast University Interdisciplinary Research Program for Young Scholars, Guangxi Science and Technology Achievement Transformation Program No.ZG2504240019, Science and Technology Major Special Program of Jiangsu Grants No. BG2024028, the Natural Science Foundation of Jiangsu Province under Grant No. BK20253020, BK20230083, SEU Innovation Capability Enhancement Plan for Doctoral Students CXJH_SEU 26020, and in part by the Collaborative Innovation Center of Novel Software Technology and Industrialization.

References

- [1] Aleksandar Bojchevski and Stephan Günnemann. 2017. Deep gaussian embedding of graphs: Unsupervised inductive learning via ranking. *arXiv preprint arXiv:1707.03815* (2017).
- [2] Nicholas Carlini and David Wagner. 2017. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)*. Ieee, 39–57.
- [3] Linbo Chen, Dewen Lu, Keke Sun, Yuemei Xu, Pingping Hu, Xianpeng Li, and Feng Xu. 2019. Identification of biomarkers associated with diagnosis and prognosis of colorectal cancer patients based on integrated bioinformatics analysis. *Gene* 692 (2019), 119–125.
- [4] Wei Chen, Chi Wang, and Yajun Wang. 2010. Scalable influence maximization for prevalent viral marketing in large-scale social networks. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1029–1038.
- [5] Chia-Hao Chin, Shu-Hwa Chen, Hsin-Hung Wu, Chin-Wen Ho, Ming-Tat Ko, and Chung-Yen Lin. 2014. cytoHubba: identifying hub objects and sub-networks from complex interactome. *BMC systems biology* 8, 4 (2014), 1–7.
- [6] Piotr Dabkowski and Yarin Gal. 2017. Real time image saliency for black box classifiers. *Advances in neural information processing systems* 30 (2017).
- [7] Christian Etmann, Sebastian Lunz, Peter Maass, and Carola-Bibiane Schönlieb. 2019. On the connection between adversarial robustness and saliency map interpretability. *arXiv preprint arXiv:1905.04172* (2019).
- [8] Ronald Aylmer Fisher. 1992. Statistical methods for research workers. In *Breakthroughs in statistics*. Springer, 66–70.
- [9] Ruth C Fong and Andrea Vedaldi. 2017. Interpretable explanations of black boxes by meaningful perturbation. In *Proceedings of the IEEE international conference on computer vision*. 3429–3437.
- [10] Thorben Funke, Megha Khosla, and Avishek Anand. 2020. Hard masking for explaining graph neural networks. (2020).
- [11] Ruize Gao, Jiong Xiao Wang, Kaiwen Zhou, Feng Liu, Binghui Xie, Gang Niu, Bo Han, and James Cheng. 2022. Fast and reliable evaluation of adversarial robustness with minimum-margin attack. In *International Conference on Machine Learning*. PMLR, 7144–7163.
- [12] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572* (2014).
- [13] Sven Gowal, Jonathan Uesato, Chongli Qin, Po-Sen Huang, Timothy Mann, and Pushmeet Kohli. 2019. An alternative surrogate loss for pgd-based adversarial testing. *arXiv preprint arXiv:1910.09338* (2019).
- [14] Steven A Greenberg. 2009. How citation distortions create unfounded authority: analysis of a citation network. *Bmj* 339 (2009).
- [15] William L Hamilton, Rex Ying, and Jure Leskovec. 2017. Inductive representation learning on large graphs. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*. 1025–1035.
- [16] Yi-Qi Hu, Hong Qian, and Yang Yu. 2017. Sequential classification-based optimization for direct policy search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 31.
- [17] Qiang Huang, Makoto Yamada, Yuan Tian, Dinesh Singh, and Yi Chang. 2022. Graphlime: Local interpretable model explanations for graph neural networks. *IEEE Transactions on Knowledge and Data Engineering* (2022).
- [18] Matthew O Jackson. 2010. *Social and economic networks*. Princeton university press.
- [19] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
- [20] Thai Le, Suhang Wang, and Dongwon Lee. 2020. GRACE: Generating Concise and Informative Contrastive Sample to Explain Neural Network Model's Prediction. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 238–248.
- [21] Yu-Ren Liu, Yi-Qi Hu, Hong Qian, Yang Yu, and Chao Qian. 2017. Zoopt: Toolbox for derivative-free optimization. *arXiv preprint arXiv:1801.00329* (2017).
- [22] Dongsheng Luo, Wei Cheng, Dongkuan Xu, Wenchao Yu, Bo Zong, Haifeng Chen, and Xiang Zhang. 2020. Parameterized explainer for graph neural network. *Advances in neural information processing systems* 33 (2020), 19620–19631.
- [23] Pierre May and Evelyn May. 1999. Twenty years of p53 research: structural and functional aspects of the p53 protein. *Oncogene* 18, 53 (1999), 7621–7636.
- [24] Malaak Nasser Moussa, Crystal D Vechlekar, Jonathan H Burdette, Matt R Steen, Christina E Hugenschmidt, and Paul J Laurienti. 2011. Changes in cognitive state alter human functional brain networks. *Frontiers in human neuroscience* 5 (2011), 83.
- [25] Steven A Narod and William D Foulkes. 2004. BRCA1 and BRCA2: 1994 and beyond. *Nature Reviews Cancer* 4, 9 (2004), 665–676.
- [26] Phillip E Pope, Soheil Kolouri, Mohammad Rostami, Charles E Martin, and Heiko Hoffmann. 2019. Explainability methods for graph convolutional neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 10772–10781.
- [27] Chao Qian, Yang Yu, and Zhi-Hua Zhou. 2015. Subset selection by Pareto optimization. *Advances in neural information processing systems* 28 (2015).
- [28] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why should I trust you?" Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 1135–1144.
- [29] Matthew Richardson and Pedro Domingos. 2002. Mining knowledge-sharing sites for viral marketing. In *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. 61–70.
- [30] Matthew E Ritchie, Belinda Phipson, DI Wu, Yifang Hu, Charity W Law, Wei Shi, and Gordon K Smyth. 2015. limma powers differential expression analyses for RNA-sequencing and microarray studies. *Nucleic acids research* 43, 7 (2015), e47–e47.
- [31] Michael Sejr Schlichtkrull, Nicola De Cao, and Ivan Titov. 2020. Interpreting graph neural networks for nlp with differentiable edge masking. *arXiv preprint arXiv:2010.00577* (2020).
- [32] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. 2017. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*. 618–626.
- [33] Amir Sheikhanmadi, Mohammad Ali Nematbakhsh, and Arman Shokrollahi. 2015. Improving detection of influential nodes in complex networks. *Physica A: Statistical Mechanics and its Applications* 436 (2015), 833–845.
- [34] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. 2013. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034* (2013).
- [35] Daniel Smilkov, Nikhil Thorat, Been Kim, Fernanda Viégas, and Martin Wattenberg. 2017. Smoothgrad: removing noise by adding noise. *arXiv preprint arXiv:1706.03825* (2017).
- [36] Olaf Sporns. 2018. Graph theory methods: applications in brain networks. *Dialogues in clinical neuroscience* 20, 2 (2018), 111.
- [37] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
- [38] Sandra Wachter, Brent Mittelstadt, and Chris Russell. 2017. Counterfactual explanations without opening the black box: Automated decisions and the GDPR. *Harv. JL & Tech.* 31 (2017), 841.
- [39] Haoyu Peter Wang, Nan Wu, Hang Yang, Cong Hao, and Pan Li. 2022. Unsupervised Learning for Combinatorial Optimization with Principled Objective Relaxation. In *Advances in Neural Information Processing Systems*.
- [40] Yuanzhi Yang, Lei Yu, Xing Wang, Zhongliang Zhou, You Chen, and Tian Kou. 2019. A novel method to evaluate node importance in complex networks. *Physica A: Statistical Mechanics and its Applications* 526 (2019), 121118.
- [41] Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik, and Jure Leskovec. 2019. Gnnexplainer: Generating explanations for graph neural networks. *Advances in neural information processing systems* 32 (2019), 9240.
- [42] Jianxiang Yu, Yuxiang Ren, Chenghua Gong, Jiaqi Tan, Xiang Li, and Xuechang Zhang. 2025. Leveraging large language models for node generation in few-shot learning on text-attributed graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 13087–13095.
- [43] Hao Yuan, Lei Cai, Xia Hu, Jie Wang, and Shuiwang Ji. 2020. Interpreting image classifiers by generating discrete masks. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2020).
- [44] Hao Yuan, Haiyang Yu, Shurui Gui, and Shuiwang Ji. 2020. Explainability in graph neural networks: A taxonomic survey. *arXiv preprint arXiv:2012.15445* (2020).
- [45] Baoliang Zhang, Xiaoxin Guo, Zhenchuan Tu, and Jia Zhang. 2022. Graph alternate learning for robust graph neural networks in node classification. *Neural Computing and Applications* 34, 11 (2022), 8723–8735.
- [46] Yuxin Zhao, Shenghong Li, and Feng Jin. 2016. Identification of influential nodes in social networks with community structure based on label propagation. *Neurocomputing* 210 (2016), 34–44.
- [47] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2020. Graph neural networks: A review of methods and applications. *AI Open* 1 (2020), 57–81.

A APPENDIX

A.1 Supplementary Method

A.1.1 Details of GNN Model. Since our goal is estimating GNN model to fit the graph data rather than pursuing the state-of-the-art classification accuracy, we employ the well-established GCN [19] without dropout or other tricks. $H^{(l)}$ is the node representation of layer l , in the first layer we have $H^{(0)} = X$. The hidden layer $l + 1$ is defined as

$$H^{(l+1)} = \sigma(\hat{A}H^{(l)}W^{(l)}), \quad (14)$$

where $\hat{A} = \tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}$ is the symmetric normalized adjacency matrix, $\tilde{A} = A + I_n$ is the adjacency matrix adding self-loops with identity matrix I_n , and $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$ is the degree matrix. $\sigma(\cdot)$ is an activation function (usually ReLU). $W^{(l)}$ is a trainable parameter of layer l .

Following [19], we consider a GCN with a single hidden layer:

$$\mathbf{P}^{\mathcal{G}} = \text{softmax}\left(\hat{A}\sigma(\hat{A}XW^{(1)})W^{(2)}\right), \quad (15)$$

where $\mathbf{P}^{\mathcal{G}} \in \mathbb{R}^{n \times m}$ is the probabilistic matrix, and $\mathbf{P}_v^{\mathcal{G}} \in \mathbb{R}^n$ is the probability distribution on class labels for node v and $\mathbf{P}_{v,c}^{\mathcal{G}} \in \mathbb{R}$ represents the probability on class $c \in C$ for node v . The model parameter $W^{(1)}, W^{(2)}$ is optimized using the cross-entropy loss:

$$\mathcal{L} = \sum_{v \in \mathcal{V}} \sum_c \left(1(\mathbf{Y}_v^{\mathcal{G}} = c) \cdot \ln \mathbf{P}_{v,c}^{\mathcal{G}}\right), \quad (16)$$

Then we get the fitting result

$$\hat{\mathbf{Y}}_v^{\mathcal{G}} = \underset{c}{\operatorname{argmax}} \mathbf{P}_{v,c}^{\mathcal{G}} \quad (17)$$

Intuitively, the influential nodes selected should be independent of the specific GNN model used, as different GNN models are based on similar aggregation principles. **We adopt the Graph Convolutional Network (GCN) due to its balance of power and robustness; more complex models often exhibit less robustness.** Empirical evidence [45] supports this choice: nodes identified as influential through GCN perturbation also significantly impact predictions when using other models like Graph Attention Networks (GAT). This cross-model consistency demonstrates the robustness and general applicability of our approach.

A.1.2 Variations of Fidelity Formulation. There are various choices to approximate absolute prediction change rather than CE-softmax in (4). [2] pointed out the phenomenon of gradient vanishing in the widely used CE loss, and it further proposed Carlili-Wagner (CW) loss.

$$\mathcal{L}_{\text{CW}}(\mathbf{s}) = \sum_{v \in \mathcal{V}} \left(-\max_{c \neq \hat{\mathbf{Y}}_v^{\mathcal{G}}} \ln \mathbf{P}_{v,c}^{\mathcal{G}'} + \ln \mathbf{P}_{v,\hat{\mathbf{Y}}_v^{\mathcal{G}}}^{\mathcal{G}'}\right). \quad (18)$$

We aim to classify each node v as another whatever wrong class and has maximal margin the original predicted class $\hat{\mathbf{Y}}_v^{\mathcal{G}}$. Since the wrong class is not targeted, the class with the largest probability $\max_{c \neq \hat{\mathbf{Y}}_v^{\mathcal{G}}} \mathbf{P}_{v,c}^{\mathcal{G}'}$ is the optimal choice. However, this is infeasible for GGUN, since the maximize operation is non-differential, which is necessary for derivative-based optimization. Although derivative-free optimization does not require the score function to be differentiable, the queries for the objective function will incur a computational cost. For instance-level perturbation, it is acceptable to traverse all classes and select the maximal probability class for a specific node. However, for our global perturbation goal, which

requires a traverse of all nodes in the large graph, the computational cost becomes prohibitive. A natural intuition is that the second-best class (the class with the greatest probability other than the predicted class $\hat{\mathbf{Y}}_v^{\mathcal{G}}$) on the original graph, $\tilde{\mathbf{Y}}_v^{\mathcal{G}} = \operatorname{argmax}_{c \neq \hat{\mathbf{Y}}_v^{\mathcal{G}}} \mathbf{P}_{v,c}^{\mathcal{G}}$, is most likely to be the targeted wrong class [11, 13]:

$$\max_{c \neq \hat{\mathbf{Y}}_v^{\mathcal{G}}} \mathbf{P}_{v,c}^{\mathcal{G}} = \mathbf{P}_{v,\tilde{\mathbf{Y}}_v^{\mathcal{G}}}^{\mathcal{G}}. \quad (19)$$

It is worth noting that both the predicted class $\hat{\mathbf{Y}}^{\mathcal{G}}$ and the second-best class $\tilde{\mathbf{Y}}^{\mathcal{G}}$ can be pre-computed before perturbation optimization, when the original model is being trained, through traversing the probabilistic matrix $\mathbf{P}^{\mathcal{G}}$ once. In this way, the maximize operation is replaced by a quick lookup operation on $\tilde{\mathbf{Y}}^{\mathcal{G}}$:

$$\mathcal{L}_{\text{CW-logSoftmax}}(\mathbf{s}) = \sum_{v \in \mathcal{V}} \left(-\ln \mathbf{P}_{v,\tilde{\mathbf{Y}}_v^{\mathcal{G}}}^{\mathcal{G}'} + \ln \mathbf{P}_{v,\hat{\mathbf{Y}}_v^{\mathcal{G}}}^{\mathcal{G}'}\right) \quad (20)$$

During GNN training, we use CE loss, which takes the logarithm of the output probability matrix as shown in (16). For implementation, we directly use logSoftmax operator to calculate Cross Entropy at one time to avoid the problem of overflow. When optimizing input on the fixed model, the logarithm operation might not be necessary. Thus, we propose 2 additional versions (21a) and (21b) by removing logarithm from CE-logSoftmax and CW-logSoftmax:

$$\mathcal{L}_{\text{CE-softmax}}(\mathbf{s}) = \sum_{v \in \mathcal{V}} \mathbf{P}_{v,\tilde{\mathbf{Y}}_v^{\mathcal{G}}}^{\mathcal{G}'} \quad (21a)$$

$$\mathcal{L}_{\text{CW-logSoftmax}}(\mathbf{s}) = \sum_{v \in \mathcal{V}} \left(-\mathbf{P}_{v,\tilde{\mathbf{Y}}_v^{\mathcal{G}}}^{\mathcal{G}'} + \mathbf{P}_{v,\hat{\mathbf{Y}}_v^{\mathcal{G}}}^{\mathcal{G}'}\right), \quad (21b)$$

Overall, we propose four variations of the score function as an approximation of the non-differential (4). In the experiment, we use (4) as the evaluation metric to compare the performance of each approximation.

A.2 Supplementary Experimental Settings And Results In Claim 1

A.2.1 Baseline methods: Baseline methods can be divided into groups. Centrality-based methods contain traditional metrics that only consider graph structure. GNNExplainer, a well-established node-specific explanation method, selects only important features/edges for specific nodes (local interpretability). PGExplainer, a parameterized explainer for GNNs, adopts a deep neural network to parameterize the generation process of explanations. GraphLIME, a local interpretable model explanation for graphs using the Hilbert-Schmidt Independence Criterion (HSIC) Lasso. Gradient and Greedy share a common objective function with GGUN in (21a), with different optimization algorithms. EHCC introduces an extended degree to refine classical degree centrality, using E-shell decomposition to position nodes hierarchically. KPDN combines local and global features with a fusion degree K-shell and the Solton index to better capture node influence and relationships. For fairness, we select K influential nodes for each method: (1) Centrality: We choose 3 widely-used node centrality metrics: Degree, Eigenvector, and Closeness. We rank the centrality and choose the top- K nodes. (2) GNNExplainer: A node-specific explanation method. For a fair comparison, we take a traverse strategy: we first obtain an explanation for each node, then we rank the selected edges and choose the top- K attached nodes. (3) PGExplainer: A parameterized explainer

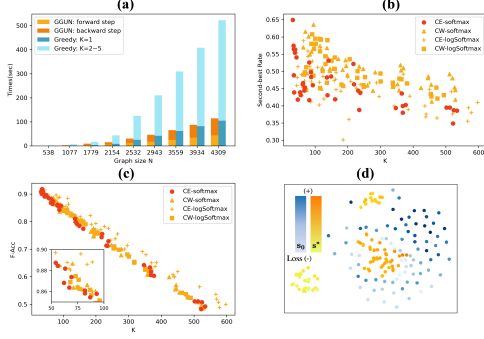


Figure 6: (a) Time v.s. the graph size N . For greedy, the time complexity is linear with K , thus we only record the time from $K = 1$ to $K = 5$. Greedy search selecting one node takes almost the same time as GGUN selecting all nodes simultaneously. (b) Comparison of the second-best rate for four variations of differential approximation. CW has a higher Second-best Rate compared with CE regardless of the use of logarithms. (c) Comparison of F1 for 4 variations of differential approximation of (4). For small K values ($K \in [50, 100]$), CE-softmax slightly outperforms the other three variations.

for GNNs. For a fair comparison, we take a traverse strategy as GNNExplainer. (4) GraphLIME: A nonlinear feature selection method. For a fair comparison, we take a traverse strategy as GNNExplainer. (5) Gradient: It directly takes a single step of the gradient of the score function on adjacency matrix: $\nabla_A \mathcal{L}$ and sums up the absolute value for each row and column $\|\nabla_A \mathcal{L}_{:,i}\|_1 + \|\nabla_A \mathcal{L}_{i,:}\|_1$, then rank them to get the top- K nodes. (6) Greedy: An intuitive solution to our originally formulated CO problem in (4). For each step, we remove each node from the graph and record the largest response node, and add it to S , until $|S| = K$. (7) EHCC: The hybrid characteristic centrality combines the extended degree and the E-shell decomposition. We computed the extended degree of each node and the position index from the E-shell decomposition and selected the top- K nodes based on their hybrid characteristic centrality. (8) KPDN: It integrates local and global features, using a fusion degree K-shell decomposition to enhance node discrimination and Solton index to capture the association between a node and its neighbors. We rank the importance of nodes and select the top- K nodes.

A.2.2 Implementation details. We first estimate a two-layer GCN model with hidden dimension $d = 16$ on the original graph to obtain optimal parameters and the probabilistic matrix P^G . Then we employ GGUN to select the group of influential nodes S . We compare GGUN with baselines on three GNN models: GCN-fixed (the model with fixed parameters after training on the original graph), GCN-new (another trained model), and GAT[37]. The number of training epochs is set to 1000. For the following experiments, we average the results of 5 seeds ranging from 42 to 46. We implement our method in Pytorch and execute all experiments on NVIDIA RTX A6000 GPUs.

A.2.3 Complexity analysis. We have conducted a rough analysis of the computational complexity of greedy search and GGUN, but when it comes to the complexity of the neural network’s forward and backpropagation, it cannot be theoretically analyzed and requires experimental analysis.

We use the FakeDataset class in Pytorch to generate graphs with different sizes n , and analyze the efficiency of GGUN and Greedy to solve the combinatorial optimization problem, with increasing size of the graph. We fix the node feature dimension $d = 128$, and the class number $c = 2$. For fairness, we implement our method and greedy search, conducting the complexity analysis experiments on CPU.

For greedy search, since the time complexity is linear with K , we record the time for $K = 1$ and $K = 5$. We can see that the greedy method takes almost the same time as GGUN when selecting only one node. With increasing K , the time cost will increase linearly, while for GGUN, all nodes are selected simultaneously. We record the forward and backward time of GGUN with 100 steps. As shown in Fig. 6(a), the backward step takes more time than the forward step. More comparison experiments on the complexity analysis are omitted due to space limitations.

A.2.4 Optimization analysis. We evaluate the performance of both derivative-based and derivative-free optimization methods. For derivative-based, we utilize PGD with a learning rate of $\eta = 0.1$ and 100 iterations. To account for the sensitivity of non-convex objectives to the initial point, we randomly sample k_1 initial points from $[0, 1]^n$ and select the best one. For derivative-free optimization, we use the well-established toolkit ZOOpt[21] with the built-in algorithm SPACOS[16]. The performance depends on the input hyperparameter k_2 , which represents the comparison times. Both methods show improved optimization performance with larger values of k_1 or k_2 , but at the expense of increased computation cost. To make a fair comparison, we evaluate the optimized loss with the time cost maintained constant.

The results in Table 6 indicate that PGD outperforms SPACOS in terms of lower optimized loss. As SPACOS enables efficient approximation of global optimal solutions for arbitrary functions, lower loss implies the capability of PGD to overcome the local optimum problem with suitable hyperparameters.

Then we employ the best solution from PGD (s^* with a loss of 2419.33) to analyze two different recovery algorithms: Hard Threshold and Compared Rounding. As shown in Table 7, both algorithms increase the loss from s^* , while Compared Rounding achieves lower loss. Such observation is attributed to the belief that Compared Rounding rounds entries in s^* to either 0 or 1 based on optimization objectives, yet with additional computational cost.

Table 6: Comparison of optimization loss ($\downarrow$) between Derivation-based (PGD) and Derivation-free (SPACOS) optimization with equal time cost.

Optimization	150s	1500s	3000s
PGD	2440.92	2426.10	2419.33
SPACOS	3672.29	2744.32	2666.00

A.2.5 Variations of methods. We seek to validate if CW still outperforms CE as previously established works in our problem settings. The comparison results are revealed in Fig. 6 (b). Unexpectedly, CW underperforms CE, which is inconsistent with the previous statement. When K is small ($K \in [50, 100]$), CE-softmax even holds slight advantages over CW-softmax. The reliability of the Second-best approximation (which can be verified by calculating the Second-best

Table 7: Comparison of the performance of two recovery algorithms. Compared Rounding obviously outperforms Hard Threshold with lower loss, yet an obvious time cost increment.

Rounding	Loss(↓)	Time(↓)
s^*	2419.33	—
Hard Threshold	2569.75	0.16
Compared Rounding	2486.09	82.34

Rate, see supplementary) implies that the cause of CW’s under-performance is not from the approximation, but CW itself. Such reflection is contrary to previous claims, and we suspect the reasons to be: CW only suits instance-level perturbation, i.e., changing the prediction of one instance. For our goal of globally changing predictions, it becomes difficult to optimize the accumulated probability margin of all nodes. Another factor is the utilization of the logarithm. For both CE and CW, using Softmax function is more effective than logSoftmax. In conclusion, we propose CE-softmax as the default choice for GGUN objective function.

A.2.6 Verification of Second-best assumption. As shown above, we propose 4 variations for approximating the original non-differential fidelity. For CW, the maximization operation is still nondifferential. Natural intuition is that the second-best class (the class with the greatest probability other than the predicted class $\hat{Y}_v^G$) on the original graph, $\hat{Y}_v^G = \arg\max_{c \neq \hat{Y}_v^G} P_{v,c}^G$, is most likely to be the targeted wrong class:

$$\max_{c \neq \hat{Y}_v^G} P_{v,c}^{G'} = P_{v, \hat{Y}_v^G}^G. \quad (22)$$

We aim to verify this assumption by calculating the Second-best Rate, i.e., misclassification into the second-best class $\hat{Y}_v^G$ as a percentage of misclassification: $\sum_{v \in \mathcal{V}} 1(\arg\max_c P_{v,c}^{G'} = \hat{Y}_v^G)$. As shown in Fig.6(b), the second-best rate of CW is higher than CE, both with and without the use of logarithm, confirming the belief that the Second-best class is the most probable target of misleading classification.

A.2.7 Gradient-based(PGD) Optimization Analysis. We look at the continuous optimization problem after relaxing the discrete variable ((5) in the main paper):

$$\arg\min_{s \in [0,1]^n} \mathcal{L}_{GGUN}(s) := \mathcal{L}_{CE-\logSoftmax}(s) + \lambda \|s\|_1, \quad (23)$$

We assume this objective function, which is high-dimensional and non-convex, may have many local optima, thus derivative(gradient)-based optimization methods cannot guarantee that the final solution is the global optimum. The analysis of high-dimensional non-convex functions is challenging, we conduct a case study in a 10-dimensional setting to gain insights into the optimization performance guarantees. The dataset generation is in line with the synthetic dataset: $n = 10$, $m/n = \{1/5, 1/5\}$, $e = 0$, with totally $2^{10} = 1024$ points in the binary space.

We use tSNE to visualize the optimization performance by plotting the initial points s_0 sampling from the whole $[0, 1]^N$ space, and optimal points s^* in 2-dimensional space. As shown in Fig. 6 (d), the two bars represent the loss at the corresponding points. This is also the case for the figures in the following experiments.

A.3 Other Experiments

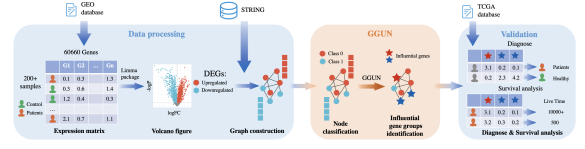


Figure 7: The workflow of Protein-Protein Interaction (PPI) influential genes detection. The first two steps of data processing and the last step of validation are the same for baselines and GGUN. For GGUN, we estimate a GNN node classification model and perturb to identify the influential groups.

It is worth noting that, the prerequisite to verify Claim 2 is to satisfy Claim 1. The foundation of data-driven methods is that GNN is capable of mining underlying information in attributed graphs: perturbing ground-truth influential features can change the GNN prediction significantly. GGUN still achieves superior performance compared to the baselines in terms of F-Prob and F-Acc on the dataset we generated in Section 5.2. For brevity, we omit several supplementary experiments from the appendix, including optimization behavior analysis, scalability studies, and synthetic validations.

In Table 8 and 9, we evaluate all the methods on a much larger graph (PubMed dataset, 20k nodes). GGUN-Hard Threshold achieves average relative improvements of 25.79% (F-Prob) and 21.71% (F-Acc) over the mean performance of all baselines except the Greedy algorithm, with similar gains observed for Compared Rounding.

Table 8: Comparison of F-Acc, F-Prob on PubMed. For GGUN, the selected group size K changed with different λ . In the cases below, we unify $K = 636, 892, 1260, 1665$ for a fair comparison with baselines.

Metric	$\lambda = 7, K = 636$		$\lambda = 6, K = 892$		$\lambda = 5, K = 1260$		$\lambda = 4, K = 1665$	
	F-Prob(↓)	F-Acc(↓)	F-Prob(↓)	F-Acc(↓)	F-Prob(↓)	F-Acc(↓)	F-Prob(↓)	F-Acc(↓)
Random	0.9226	0.9527	0.8822	0.9367	0.8582	0.9086	0.8371	0.8866
Degree	0.7477	0.7960	0.6795	0.7297	0.5939	0.6451	0.5926	0.5619
Eigenvector	0.8667	0.9119	0.8411	0.8826	0.8157	0.8532	0.7914	0.8270
Closeness	0.8046	0.8401	0.7711	0.8032	0.7269	0.7546	0.6874	0.7104
GNNExplainer-TE	0.8270	0.8787	0.7886	0.8380	0.7320	0.7837	0.6721	0.7310
Gradient	0.7171	0.7564	0.6520	0.6900	0.5650	0.6053	0.4838	0.5245
EHCC	0.8031	0.8440	0.7596	0.7972	0.6984	0.7305	0.6320	0.6588
KPDN	0.9152	0.9715	0.9060	0.9603	0.8911	0.9461	0.9756	0.9301
Greedy	0.6652	0.7267	0.5911	0.6548	0.5015	0.5661	0.4138	0.4792
GGUN-Hard Thresholding	0.6760	0.7327	0.6011	0.6637	0.5101	0.5728	0.4224	0.4878

Table 9: Comparison of F-Acc, F-Prob on PubMed. For GGUN, the selected group size K changed with different λ . In the cases below, we unify $K = 1192, 1253, 1347, 1447$ for a fair comparison with baselines.

Metric	$\lambda = 7, K = 1192$		$\lambda = 6, K = 1253$		$\lambda = 5, K = 1347$		$\lambda = 4, K = 1447$	
	F-Prob(↓)	F-Acc(↓)	F-Prob(↓)	F-Acc(↓)	F-Prob(↓)	F-Acc(↓)	F-Prob(↓)	F-Acc(↓)
Random	0.8604	0.9100	0.8598	0.9102	0.8593	0.9122	0.8474	0.8949
Degree	0.6101	0.6609	0.5964	0.6476	0.5745	0.6290	0.5528	0.6075
Eigenvector	0.8184	0.8566	0.8159	0.8533	0.8105	0.8478	0.8028	0.8387
Closeness	0.7349	0.7622	0.7285	0.7566	0.7195	0.7464	0.7090	0.7347
GNNExplainer-TE	0.7390	0.7906	0.7326	0.7843	0.7182	0.7728	0.7035	0.7584
Gradient	0.5808	0.6204	0.5664	0.6069	0.5447	0.5858	0.5249	0.5660
EHCC	0.7079	0.7409	0.6998	0.7320	0.6845	0.7161	0.6685	0.6990
KPDN	0.8939	0.9486	0.8913	0.9464	0.8876	0.9422	0.8839	0.9387
Greedy	0.5184	0.5822	0.5032	0.5677	0.4805	0.5444	0.4586	0.5207
GGUN-Compared Rounding	0.5233	0.5876	0.5063	0.5702	0.4863	0.5491	0.4657	0.5296